19

DISTANCE IN KNN (OTHER DISTANCE METRICS)

Hamming Distance– It is tailored for string comparison, especially in the context of binary data (e.g., 0s and 1s). It quantifies **dissimilarity** by counting differing symbol positions (bits) between two sequences. This metric is particularly well-suited for binary classification tasks and when categorical attributes are encoded as binary strings, such as through one-hot encoding.

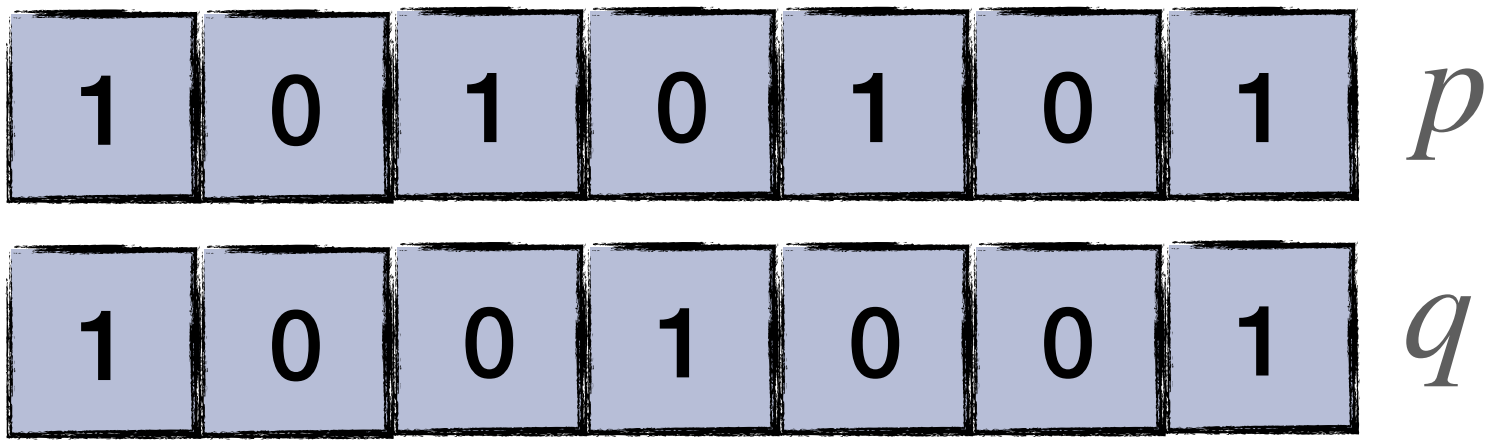
- Formula: $d(p, q) = \sum_{i=1}^{n} |p_i - q_i|$, where p_i and q_i can take values 0 or 1.

- Python Snippet:

```python
import numpy as np

p = np.array([1, 0, 1, 0, 1, 0, 1])
q = np.array([1, 0, 0, 1, 0, 0, 1])

hamming_dist = np.sum(p != q)
print("Hamming Distance:", hamming_dist)
#or
knn = KNeighborsClassifier(n_neighbors=3, metric='hamming')
```


1 0 1 0 1 0 1
p
1 0 0 1 0 0 1
q

$$d(p, q) = 3$$

DISTANCE IN KNN (OTHER DISTANCE METRICS)

Hamming Distance– It is tailored for string comparison, especially in the context of binary data (e.g., 0s and 1s). It quantifies **dissimilarity** by counting differing symbol positions (bits) between two sequences. This metric is particularly well-suited for binary classification tasks and when categorical attributes are encoded as binary strings, such as through one-hot encoding.

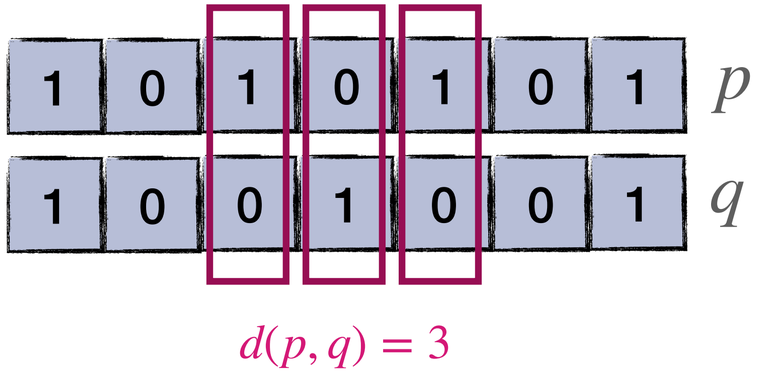
- Formula: $d(p,q) = \sum_{i=1}^{n} |p_i - q_i|$, where p_i and q_i can take values 0 or 1.

- Python Snippet:

```python
import numpy as np

p = np.array([1, 0, 1, 0, 1, 0, 1])
q = np.array([1, 0, 0, 1, 0, 0, 1])

hamming_dist = np.sum(p != q)
print("Hamming Distance:", hamming_dist)
#or
knn = KNeighborsClassifier(n_neighbors=3, metric='hamming')
```



DISTANCE IN KNN (OTHER DISTANCE METRICS)

Minkowski Distance– It is a generalized metric form. Depending on the value of a parameter r, it can represent the Euclidean distance (for $r = 2$) or the Manhattan distance (for $r = 1$). For other values of r, it can be used to capture various other distance concepts.

- Formula: $d(p, q) = \left(\sum_{i=1}^{n} |p_i - q_i|^r \right)^{1/r}$

- Python Snippet:

```
from scipy.spatial import distance
dst = distance.minkowski(p, q, p=r)

#or

knn = KNeighborsClassifier(n_neighbors=3, metric='minkowski', p=... )
# You can adjust the value of p
```